

When Systems Take Initiative: A Design Framework for Adaptive, Mixed-initiative Database Querying

Longfei Chen
School of Information Science and
Technology
ShanghaiTech University
Shanghai, China
chenlf@shanghaitech.edu.cn

Shenghan Gao
School of Information Science and
Technology
ShanghaiTech University
Shanghai, China
gaoshh1@gmail.com

Shiwei Wang
Shenzhen Yishi Huolala Technology
Co., Ltd
Shenzhen, China
three.wang@huolala.cn

Ken Lin
Shenzhen Yishi Huolala Technology
Co., Ltd
Shenzhen, China
adam.lin@huolala.cn

Yun Wang
Microsoft Research
Hong Kong, China
wangyun@microsoft.com

Yan Lu
Microsoft Research Asia
Beijing, China
yanlu@microsoft.com

Quan Li*
School of Information Science and
Technology
ShanghaiTech University
Shanghai, Shanghai, China
liquan@shanghaitech.edu.cn

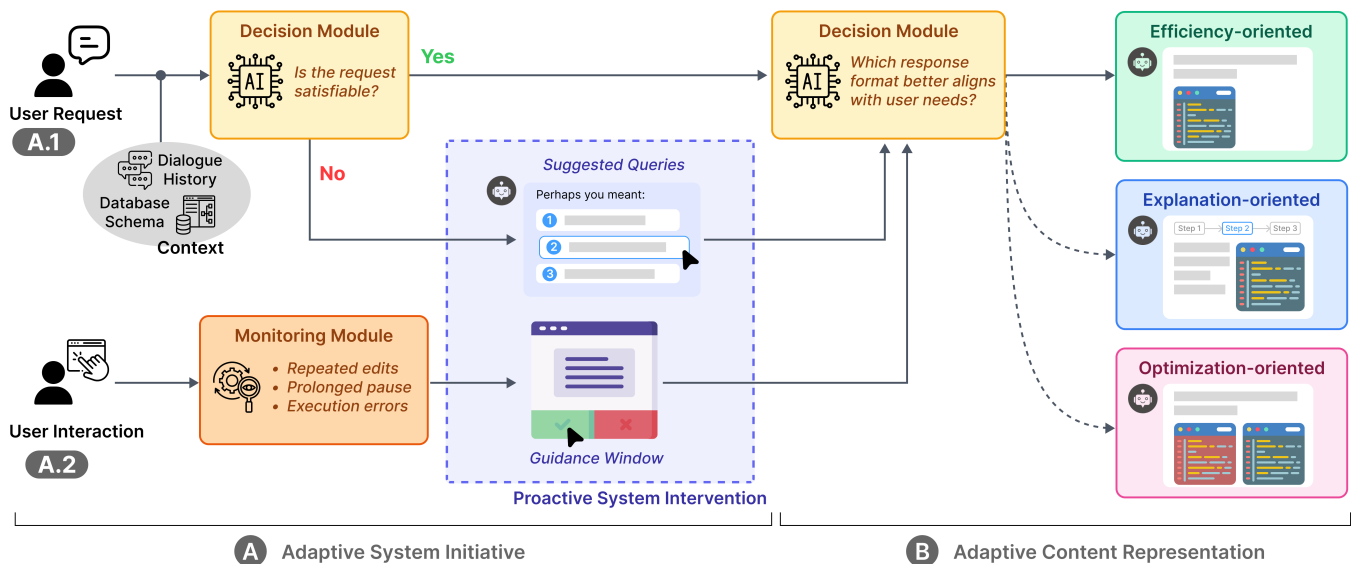


Figure 1: An adaptive, mixed-initiative database querying system that (A) strategically takes initiative based on user requests and interaction cues, and (B) dynamically adapts the representation of system-generated content to align with user needs.

*Corresponding Author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
DIS '26, Singapore, Singapore
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2563-0/26/06
<https://doi.org/10.1145/3800645.3812906>

Abstract

Exploring databases remains cognitively demanding for non-experts. While natural language interfaces offer flexibility, they place the full burden of articulation and refinement on the user, hindering exploratory discovery. We identify a core interaction design problem: how to dynamically support users' evolving understanding during query formulation. We propose and implement a paradigm of adaptive mixed-initiative interaction, where the system interprets user

behavioral cues (e.g., tentativeness, focus shifts) to infer intent and dynamically adapts its support strategies. This involves switching between responsive and proactive modes, and integrating textual responses with graphical previews to scaffold the query-building process. We instantiate this paradigm in a functional prototype. A controlled user study demonstrates that this adaptive approach not only improves task efficiency and usability but, more importantly, reduces cognitive load and fosters a more exploratory, less formulaic query-building process compared to traditional reactive interfaces.

CCS Concepts

• **Human-centered computing** → **Interactive systems and tools**; **Natural language interfaces**.

Keywords

Adaptive System, Database Querying, Human-AI Interaction

ACM Reference Format:

Longfei Chen, Shenghan Gao, Shiwei Wang, Ken Lin, Yun Wang, Yan Lu, and Quan Li. 2026. When Systems Take Initiative: A Design Framework for Adaptive, Mixed-initiative Database Querying. In *Designing Interactive Systems Conference (DIS '26)*, June 13–17, 2026, Singapore, Singapore. ACM, New York, NY, USA, 18 pages. <https://doi.org/10.1145/3800645.3812906>

1 Introduction

Database querying remains a fundamental yet cognitively demanding task, particularly for users without formal training in structured query languages (SQL). To lower this barrier, prior research [12, 20, 28, 37] has explored natural language interfaces that translate user requests into executable queries. While improving accessibility, these systems often prioritize end-to-end query generation, offering limited support for understanding how queries are constructed or why they behave in certain ways. To address this gap, interactive systems such as *DIY* [28] and *SQLucid* [37] have been proposed to support non-expert users through natural language explanations, visual representations, and interactive feedback, helping users make sense of complex query logic and increasing both query accuracy and user trust.

The emergence of large language models (LLMs) has further expanded the interaction space for database querying. Recent LLM-based database querying systems, such as *Vanna* [40] and *Dataline* [5], enable more flexible expression of user needs and provide on-demand explanations, corrections, or optimizations of generated queries. Capabilities that once required carefully designed interaction mechanisms—such as step-by-step decomposition or error repair—can now be achieved through prompting, resulting in more conversational and expressive interactions.

However, a critical limitation persists: most LLM-based systems remain fundamentally reactive. They respond to user prompts but seldom take an active role in guiding the interaction. As a result, their effectiveness heavily depends on users' ability to clearly articulate requests, diagnose issues, and formulate appropriate follow-ups. This places a considerable burden on non-expert users [47], who may lack the awareness or knowledge to assess a query's completeness or to identify necessary refinements. Unless the user explicitly asks for help, the systems provide little proactive support, shifting

the cognitive and managerial load back onto the user. This challenge is further amplified by the predominant interaction paradigm. Conversational user interfaces offer high expressive flexibility [7, 26] but often require users to mentally track complex query structures, intermediate states, and explanation threads, increasing cognitive load. Graphical user interfaces, in contrast, externalize structure and reduce memory demands but can constrain expressive freedom and require adaption to predefined interaction patterns [31, 43]. Rather than favoring one paradigm, an open design opportunity lies in creating systems that seamlessly integrate the complementary strengths of conversational and graphical interaction throughout the query formulation process.

We argue that LLM-assisted database querying for non-experts should be reconceptualized as an adaptive, mixed-initiative interaction process. Effective support should not rely solely on explicit user requests; instead, the system should be capable of inferring user needs and progress from implicit interaction cues and dynamically adjusting its strategy. When users struggle to articulate intent or assess results, the system should adopt a more proactive, guiding role; when users demonstrate confidence and clarity, it should recede into a supportive, unobtrusive mode. This shifts in focus, from merely generating correct queries to supporting continuous user understanding and decision-making, forms the core of our approach.

Building on this, we present *QueryGenie*, an adaptive interactive database querying system that interprets both users' explicit instructions and implicit interaction cues to dynamically adjust information presentation and system-user interaction. *QueryGenie* implements adaptivity along two key dimensions: (1) **Adaptive System Initiative**: The system fluidly shift between system-guided and user-driven interaction based on real-time assessment of user behavior and task context. (2) **Adaptive Content Representation**: The system dynamically coordinates conversational explanations with persistent visual representations (e.g., interactive query graphs, result previews) to externalize query logic, intermediate outcomes, and system reasoning. Through a controlled user study, we demonstrate that this adaptive, hybrid interaction model effectively combines the expressive flexibility of conversation with the cognitive benefits of graphical externalization. Compared to a state-of-the-art reactive LLM baseline, *QueryGenie* enables non-expert users to complete database querying tasks more successfully and with a significantly lower perceived cognitive load. In summary, this work makes the following contributions:

- We reconceptualize LLM-assisted database querying for non-expert users as an adaptive, mixed-initiative interaction process, critiquing the limitations of purely reactive, prompt-driven paradigms.
- We design and implement *QueryGenie*, a novel system that realizes this vision through mechanisms for adaptive initiative control and multi-modal information presentation.
- We empirically demonstrate, through a controlled user study, that our adaptive design improves both objective performance and subjective user experience, providing concrete insights for the design of future AI-assisted data interaction tools.

2 Related Work

2.1 Interactive System for Database Querying

Database querying presents a significant cognitive barrier for non-expert users. To lower this barrier, natural language interfaces to databases (NLIDB) have evolved through several paradigms. Early rule-based systems [11, 42, 48] and subsequent machine learning-based models [4, 14, 44, 49] primarily treated NL-to-SQL as an end-to-end translation task, prioritizing generation accuracy over user involvement in the query construction process.

Recognizing the limitations of this opaque, single-turn approach, the human-computer interaction (HCI) community developed interactive systems that engage users in the loop. These systems improve accuracy and understanding by enabling users to resolve ambiguities or refine outputs. One strategy employs clarification dialogues, guiding users through multiple-choice questions to disambiguate requests (e.g., *MISP* [46], *DialSQL* [9], and *PIIA* [22]). Another provide graphical tools, enabling users to map natural language to schema elements via visual controls (e.g., *NaLIR* [19], *DataTone* [8], and *DIY* [28]). More recent work, like *SQLucid* [37] lets users edit SQL by revising natural language explanations. While advancing collaboration, these systems often implement fixed, pre-defined workflows, which can constrain interaction flexibility and place the burden of process management on the user.

The advent of LLMs has profoundly reshaped the landscape. LLMs have set new benchmarks in NL-to-SQL accuracy [20] and unlocked open-ended conversational interaction. Systems like *Vanna* [40] and *Dataline* [5] allow users to fluidly request queries, explanations, or optimizations. However, their interaction paradigm remains predominantly reactive; they respond to explicit prompts but lack the capability to proactively assess user state or guide the exploration. This places a high demand on users to self-diagnose issues and articulate needs—a challenge for non-experts who may struggle to judge query completeness or identify next steps [47].

In summary, a critical gap persists between user control and system support. Prior interactive systems often impose rigid structures, while modern LLM-based interfaces offer freedom but insufficient scaffolding. What is missing is a framework that autonomously senses user intent and cognitive state to provide context-aware, adaptive support while preserving the flexibility of conversational interaction. This paper bridges that gap with *QueryGenie*, an adaptive system that uses conversational and behavioral cues to dynamically modulate its level of initiative and information presentation, fostering a more fluid and supportive human-AI partnership in data exploration.

2.2 Adaptive Interaction in Human-AI Collaboration

Adaptive interaction is a longstanding goal in HCI, describing systems that dynamically adjust their behavior, such as interface presentation, content, or level of proactivity, to align with a user's context, task, and evolving needs [15, 18]. By moving beyond static interfaces, adaptive systems aim to reduce cognitive load, improve performance, and create more fluid collaborative experiences through personalized support [1]. This adaptation can be

realized across several design dimensions: shifting the level of automation between user and system control [27, 34], tailoring the complexity of explanations to user expertise [23], or modifying conversational style to match user preferences [30, 32].

A core challenge in realizing such adaptation is the accurate sensing and modeling of the user's latent cognitive state [6, 13]. HCI research has explored a wide array of behavioral signals for this purpose. Mouse dynamics, for instance, can reveal cognitive friction and interaction patterns [29, 33]; vocal tone and speech can indicate affective states [45]; and eye gaze provides a direct measure of attentional focus and cognitive load [35, 36]. The diagnostic value of any signal, however, is highly context-dependent. Effective adaptive design therefore requires the careful selection of signals that are both practically measurable and theoretically meaningful for the specific interaction scenario and the cognitive states they intend to capture.

In this work, we apply principles of adaptive interaction to the specific challenge of NLIDB systems for non-experts. We frame query formulation not as a one-shot translation, but as a collaborative, co-constructive process between user and AI. Informed by prior work on user modeling, we identified conversational content and interaction behavior as two critical, complementary signal streams within a query-building interface. By integrating these multimodal cues, our system, *QueryGenie*, builds a real-time assessment of user need and dynamically modulates its level of initiative and form of support. This approach allows the system to move beyond a reactive role, providing context-sensitive interventions that aim to scaffold the user's sense-making process while preserving their ultimate agency over the task.

2.3 Combining Conversational and Graphical User Interfaces

Conversational user interfaces (CUIs) and graphical user interfaces (GUIs) represent complementary paradigms for human-computer interaction, each with distinct cognitive affordances [24]. CUIs, particularly when powered by LLMs, offer a low entry barrier and expressive freedom, enabling users to articulate needs in natural language, a key factor in their widespread adoption for open-ended tasks [7, 26]. However, purely text-based dialogues often lack persistent visual structure, which can increase cognitive load when reasoning about complex, stateful processes [24, 41]. In contrast, GUIs excel at externalizing system state and relationships through structured visual elements, supporting comprehension and direct manipulation [31, 43]. Yet, GUI workflows are often predefined, potentially constraining exploratory expression and requiring prior familiarity with the interface logic [3].

The synergy of these paradigms lies in their ability to address different facets of collaborative reasoning: natural language is effective for users to express what they want to achieve, while graphical representations are optimal for systems to convey how a task is structured and executed, enabling inspection and refinement [21, 39]. Prior HCI research has demonstrated the value of such hybrid designs. For example, systems like *AmbiChat* use visual disambiguation trees to ground open-domain conversations [25], while progress guides visually augment CUIs to enhance user control and self-efficacy [16].

Table 1: Participant demographics and background.

ID	Age	Gender	Occupation	Database Querying Practices
P1	27	Male	Market Analyst	Generates SQL via <i>ChatGPT/Gemini</i> and manually executes it in a cloud database environment.
P2	30	Female	Market Analyst	Generates SQL via <i>ChatGPT</i> , executes it in a cloud database, and exports results to <i>Excel</i> for analysis.
P3	25	Female	Social Science PhD	Uses <i>SQLChat</i> connected to a local database to generate and execute SQL directly within the client interface.
P4	23	Male	Data Analytics Intern	Uses <i>Copilot</i> in <i>VSCode</i> to generate SQL and insert it directly into source code files.
P5	29	Male	Financial Analyst	Generates SQL via <i>Claude</i> and manually executes it in a local database environment.
P6	34	Female	Market Analyst	Generates SQL via <i>ChatGPT</i> and manually executes it in a cloud database environment.
P7	28	Male	Product Analyst	Uses enterprise internal platform to generate and execute SQL via single-turn dialogue, without support for multi-turn dialogue.
P8	31	Male	Product Analyst	Uses enterprise internal platform to generate and execute SQL via single-turn dialogue, without support for multi-turn dialogue.

However, within the specific domain of interactive database querying, existing systems often remain polarized—adopting either a purely conversational, reactive CUI approach [5, 40], or a fixed, form-based GUI workflow [19, 28]. As argued in subsection 2.1, neither paradigm alone adequately supports the dual demands of non-expert query formulation: the need for flexible intent articulation and the need for structured cognitive externalization. This gap motivates our integrated approach, which treats conversational and graphical interaction not as separate modes, but as continuously coupled components of a unified query-building process.

3 Formative Study

To ground the design of our system in real user needs and behaviors, we conducted a formative study exploring how non-expert users approach database querying with natural language interfaces. This study aimed to understand their emergent workflows, identify points of breakdown, and uncover common strategies, thereby deriving concrete design requirements for our adaptive system.

3.1 Participants

We recruited eight participants (P1–8, 3 female, 5 male) via email and word-of-mouth. Participants were required to meet three criteria: (1) have no formal training in database query languages, (2) have a practical need to query databases in their work or studies, and (3) possess prior experience using a natural language interface for querying. Table 1 summarizes participant demographics, occupations, and self-reported experience. All participants met these criteria; notably, P3 reported using *SQLChat*¹, an open-source conversational query client, for their tasks.

3.2 Procedure

We conducted individual study sessions with each participant, each lasting approximately 50–65 minutes. Each session consisted of two phases: think-aloud walkthrough and retrospective semi-structured interview. All sessions were conducted remotely via screen sharing

and were audio-recorded and video-recorded with participants' consent.

Think-Aloud Walkthrough. We began by asking participants to demonstrate how they typically query or explore data using their preferred NL-based tool (e.g., ChatGPT, an internal NLIDB, or a BI platform with a natural language feature). To capture authentic behavior, we did not prescribe specific tasks, allowing participants to follow their own workflow and goals. Participants were asked to think aloud, verbalizing their intentions, expectations, and any points of confusion or decision-making. Our role was strictly observational; we did not guide actions or suggest strategies. We focused on how participants formulated and refined queries, interpreted results, and navigated between tools, with particular attention to moments of hesitation, breakdown, or backtracking as indicators of cognitive or interactional friction.

Retrospective Semi-Structured Interview. Following the walkthrough, we conducted a semi-structured interview to reflect on the observed interaction. Interview questions centered on participants' perceived difficulties, moments of uncertainty, and the strategies they employed to make sense of query results or assess correctness. We also asked participants to reflect on when they wished the system had provided more proactive guidance, explanations, or visual support, and when they preferred the system to remain unobtrusive. This interview phase allowed us to surface motivations, expectations, and unmet needs that were not always directly observable during interaction.

3.3 Data Analysis

We analyzed the data using thematic analysis following an inductive approach. Two researchers first independently reviewed all session recordings and performed open coding to identify initial patterns in user behaviors, challenges, and strategies. Through several rounds of discussion and affinity mapping, we iteratively grouped codes into emerging themes. These themes were then refined and validated by checking for consistency across different participants and interaction scenarios, resulting in a final set of core findings that directly informed our design goals.

¹<https://sqlchat.ai/>

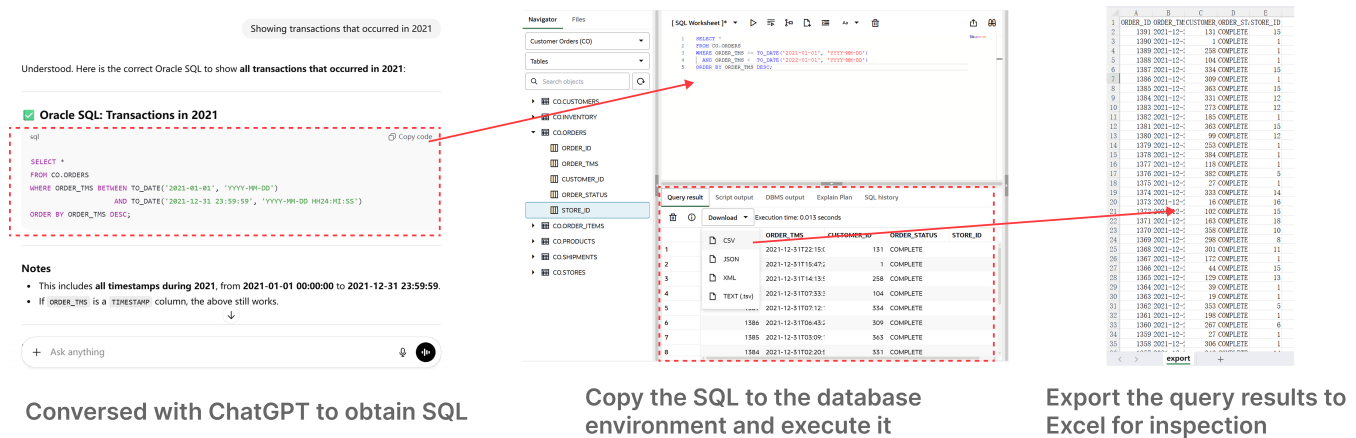


Figure 2: Case of P2: Switching among ChatGPT, the database, and Excel to construct a query.

3.4 Findings

Our analysis revealed a core tension between the iterative, sense-making nature of non-expert query construction and the interactional constraints imposed by current tools (F.1). This tension manifests in users' dynamic expectations for system initiative (F.2) and information representation (F.3) throughout the query process.

F1. Querying as Iterative Refinement Disrupted by Tool Fragmentation. We observed that query construction is rarely a linear, one-shot process. Instead, non-experts engage in a cyclical process of exploration, interpretation, and reformulation. For example, during P2's walkthrough, she began with a broad, exploratory question ("Show me the recent transactions from last month"), then incrementally narrowed the query by adding temporal constraints ("Only transactions from the last two weeks") and filtering specific attributes ("Limit this to enterprise customers in the APAC region") as she examined partial results and clarified her analytical focus. However, this sense-making process was repeatedly interrupted by tool and context switching. As shown in Figure 2, P2 had to switch across multiple platforms: she conversed with ChatGPT in the web browser to obtain SQL code, copied the generated SQL into database system to execute it, and then exported the results into Excel for inspection. As P2 reported, "This [switching between platforms] often breaks my train of thought, because I have to do extra things like copying and pasting SQL, or sending error messages back to ChatGPT when the query fails." Similar cognitive disruptions were observed in other participants' workflows. For instance, P7 and P8 both used the same internal enterprise querying platform, which treated each natural language input as an independent NL-to-SQL request, forcing them to repeatedly restate context across iterations. As P8 noted, "Sometimes I just want to make a small change, like adjusting the time window, but I still have to restate the whole question again." These breakdowns highlight how existing systems fail to support the continuity of thought essential for exploratory querying.

F2. Dynamic Expectations for Guidance Tied to Cognitive Certainty. Participants' desire for system initiative varied with their perceived clarity and control. We observed that when participants lacked a clear mental model during query construction, they

were more likely to expect the system to provide proactive guidance. This state of low cognitive certainty manifested primarily in two situations. The first occurred when users had not yet formed a concrete query goal. For example, P5 asked Claude, "I want to study recent changes in customer behavior based on this database— which tables and fields should I mainly focus on?" He explained, "I wanted it to help me figure out which data would actually be meaningful to analyze." The second situation emerged during query construction when users encountered cognitive breakdowns. For instance, P7 experienced an unexpected error while executing a SQL query to retrieve monthly sales data, which temporarily stalled his progress. He later reflected, "Because I'm not very familiar with SQL, it was hard for me to locate the problem right away. If the system could have given me some hints or assistance at that moment, it would have helped a lot." Conversely, when users had a clear mental model and direction, unsolicited intervention was seen as intrusive. This finding underscores that support needs are state-dependent, challenging the design of static interaction paradigms.

F3. The Need for Adaptive Content Representation Aligned with Intent. During query construction with conversational LLM-based tools, participants often received responses presented primarily as long, unstructured text. Such representations were not always well aligned with users' immediate query intents. Multiple participants (P1–3, P5–6) reported that interpreting purely textual outputs could impose substantial cognitive burden, particularly when they needed to understand query structures, inspect constraints, or verify intermediate decisions. As P3 noted, "When everything is in text, I have to scroll back and forth to figure out what the query is actually doing, and it's easy to miss important conditions." Our analysis further reveals that users' preferences for content representation vary dynamically with their immediate query intent. When users aim to quickly assess whether a query aligns with their analytical goals, they tend to prefer concise representations that foreground key elements. In contrast, when users seek to deeply understand a query's internal logic or examine relationships among conditions, they favor representations that are more structured and explicitly organized. P1 articulated this distinction clearly: "Sometimes I just want to quickly check whether the SQL is roughly doing

the right thing. But other times, I need to really understand how the SQL is put together. In different situations, I want the system to present the content in different ways." This points to a need for dynamic, intent-aware representations that reduce cognitive overhead.

3.5 Design Considerations

Based on our formative study findings, we distill three core design considerations (DCs) for building an adaptive, mixed-initiative system that supports collaborative database querying.

DC1. Support a Continuous Workflow for Human-AI Collaborative Query Construction. F1 shows that non-expert users often need to switch across multiple tools when constructing queries, such as conversing with an LLM to generate SQL, executing queries in a database system, and inspecting or analyzing results in separate tools. This fragmentation at the tool level frequently disrupts users' cognitive flow and interferes with their ongoing sense-making process. Therefore, our system should integrate query construction, query execution, and result inspection within a unified interaction environment. Such integration allows users to iteratively build queries through continuous dialogue with the LLM, execute SQL, and immediately examine results without leaving the workflow, thereby preserving cognitive continuity. In addition, the system should maintain contextual awareness across multiple interaction turns, treating query construction as a continuously evolving process rather than a one-shot NL-to-SQL translation task.

DC2. Modulate System Initiative Based on Inferred Cognitive State. As indicated by F2, users' expectations for system guidance vary with their underlying mental state. When users have not yet formed a clear query goal or encounter obstacles during query construction, they tend to welcome more proactive system guidance. In contrast, when users have a well-defined intent and a clear plan of action, unsolicited system interventions may be unnecessary or even disruptive. Rather than adopting a fixed level of initiative, our system should infer user cognitive certainty by integrating both explicit signals (e.g., natural language prompts) and implicit interaction cues (e.g., hesitation, prolonged inactivity, frequent backtracking, or repeated revisions). Based on these inferences, the system can appropriately modulate the timing and degree of its interventions, providing timely support without undermining user control.

DC3. Adapt Content Representations to Task Intent. As shown in F3, purely textual responses can impose additional cognitive burden, particularly when users need to understand query structures, inspect constraints, or validate intermediate decisions. Importantly, users' needs are highly context-dependent. When users aim to quickly assess whether a query broadly aligns with their intent, concise representations are often sufficient. In contrast, deeper understanding requires more structured and information-rich representations. Therefore, our system should support multiple forms of content representation and dynamically select them based on inferred user intent, enabling users to efficiently access and interpret information.

4 QueryGenie: Design and Implementation

Grounded in our design considerations (DC1–3), we developed *QueryGenie*, a system that integrates query expression, generation, execution, and result inspection into a unified interface, while proactively intervening and adapting its responses to users' evolving cognitive needs. In this section, we first describe the system interface, then detail the adaptive mechanisms that govern system initiative and content representation, and finally summarize implementation.

4.1 System Interface

Guided by DC1, *QueryGenie* addresses the frequent cognitive disruptions caused by tool fragmentation by integrating key functionalities (including query expression, generation, execution, and result inspection) across the lifecycle of human-AI collaborative query construction into a unified interaction environment. This design enables users to maintain their ongoing sense-making process within a continuous workflow.

As shown in Figure 3, the system interface consists of two primary components: (A) **Database Overview Panel**, which allows users to connect to a database and inspect its schema; and (B) **Chat Panel**, which serves as the central interaction space where users carry out multiple core activities involved in query construction:

Expressing query intent through natural language dialogue. The system supports continuous, conversational interaction, allowing users to articulate and iteratively refine their query intent using natural language (Figure 3 B.1). Across multiple turns, *QueryGenie* automatically maintains conversational context and query state, avoiding the treatment of each user input as an isolated NL-to-SQL request.

Inspecting LLM-generated content. The conversational panel presents LLM-generated outputs in the form of response bubbles, enabling users to examine and understand. For example, Figure 3 (B.2) shows a step-by-step explanation of a generated SQL query provided by the LLM. Importantly, the internal organization and presentation of content within these response bubbles is not fixed; instead, it adapts to the system's inference of the user intent. The adaptive logic and specific representation strategies are discussed in detail in subsection 4.2.

Executing SQL queries and previewing results. SQL generated by the LLM is encapsulated within executable code blocks (Figure 3 B.3). Users can directly execute a query by clicking the run button in the top-right corner of the SQL block, which runs the query against the currently connected database. The query results are then rendered immediately below the SQL block in a tabular format, allowing users to inspect outcomes without leaving the conversational context.

4.2 Adaptive Mechanism

To support system initiative (DC2) and adaptive content representation (DC3), *QueryGenie* incorporates an adaptive mechanism that continuously infers users' cognitive state during query construction and adjusts both interaction flow and system output accordingly. Figure 1 illustrates this mechanism, which integrates signals from users' explicit requests and implicit interaction behaviors to guide system decisions.

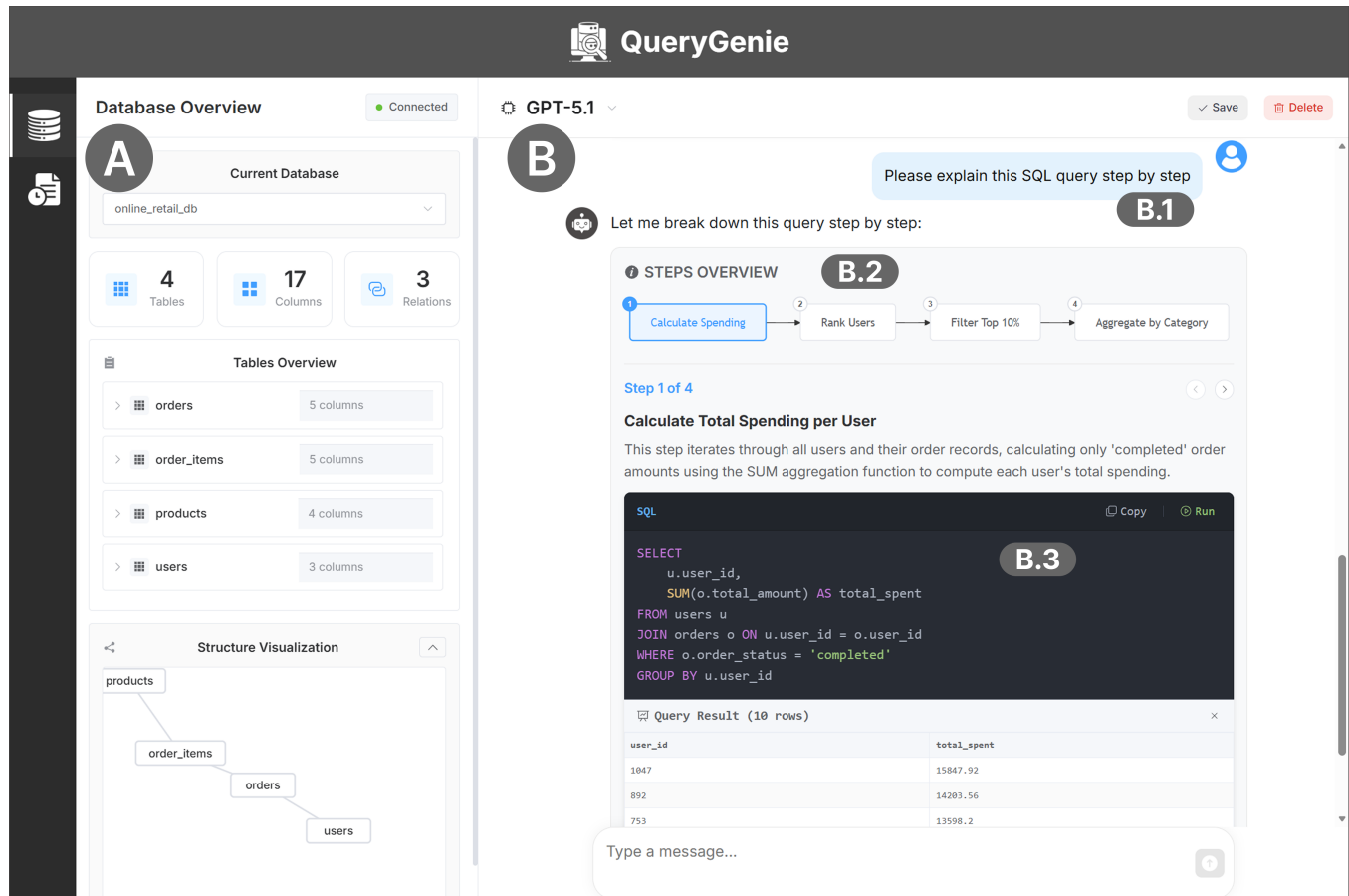


Figure 3: The system interface of QueryGenie comprises two main components: (A) Database Overview Panel, which allows users to connect to a database and inspect its schema; and (B) Chat Panel, which serves as the primary space for human-AI interaction.

4.2.1 Adaptive System Initiative. As shown in Figure 1 (A.1), when a user issues a natural language request, *QueryGenie* first interprets the request in context to identify its underlying interaction goal. Based on our design scope, the system treats user requests during query construction as belonging to one of two canonical types: (1) **requests aimed at generating a new query**, and (2) **requests aimed at iterating on an existing query**. These two types cover the primary ways users advance query construction in practice and allow the system to reason explicitly about goal feasibility.

For requests aimed at generating a new query, *QueryGenie* evaluates whether the request specifies (or allows the system to infer) a sufficiently complete set of query constraints. A request is considered satisfiable only when all of the following criteria are met:

- **Identifiable data source:** The target data tables and columns can be explicitly identified or reliably inferred.
- **Specified analytical focus:** The attributes or measures of interest (e.g., columns to be selected, aggregated, or analyzed) are provided or implied.
- **Semantically required constraints:** Any constraints that are necessary to make the query meaningful (e.g., a temporal

scope for trend analysis, aggregation targets for summaries, or grouping dimensions for comparisons) are present.

If any of these criteria cannot be satisfied based on the current context, the request is treated as unsatisfiable. For example, an initial request “show me the sales” would be judged as unsatisfiable due to the absence of a time constraint.

Similarly, for requests aimed at iterating on an existing query, *QueryGenie* determines their satisfiability based on a set of criteria:

- **Availability of a reference query:** A relevant prior query exists in the current conversational context and can serve as the basis for modification.
- **Existence of referenced elements:** Any attributes, tables, or measures referenced in the request are available in, or can be derived from, the existing query.
- **Structural compatibility:** The requested operation (e.g., filtering, comparison, aggregation, or breakdown) is compatible with the current query structure.
- **Specification of required parameters:** Any parameters necessary to apply the modification (e.g., comparison baseline,

filter value, or grouping dimension) are explicitly stated or can be inferred.

If the request fails to meet any of these criteria, it is deemed unsatisfiable. For instance, a request “*break this down by region*” is satisfiable only if a regional attribute is available in the existing query, whereas a request like “*compare this with last year*” is unsatisfiable when no temporal reference is present.

As illustrated in Figure 1 (A), when the system determines that a request is unsatisfiable, it proactively intervenes. Specifically, the system explains why the request cannot be fulfilled in its current form (e.g., missing required constraints, incompatible operations, or insufficient contextual information) and generates a set of satisfiable recommended queries. These recommendations are presented as selectable options and represent plausible next-step goals inferred from the current conversational and query context. Users can directly select one of these options to proceed, or continue the dialogue to provide additional information, thereby transitioning the query state into a satisfiable form. Figure 4 (A) shows a concrete example in which an initially unsatisfiable request is resolved by guiding the user toward a valid query through recommended options.

In addition to explicit user requests, *QueryGenie* monitors a set of implicit interaction signals to detect potential user difficulties (Figure 1 A.2). These signals are grounded in observations from the think-aloud walkthroughs conducted during our formative study. Participants frequently edited their prompts repeatedly when struggling to articulate their intent. They also exhibited prolonged inactivity after receiving system responses that were hard to interpret, particularly when outputs consisted of long, unstructured text. Furthermore, SQL execution errors (e.g., syntax errors or empty result sets) visibly stalled their progress. Building on these findings, the monitoring module tracks three behavioral signals:

- **Repeated edits to the input prompt:** The module counts how many times a user modifies the input field content before submitting a message. If a user edits the same message three or more times within a 60-second window without submission, the system infers difficulty in articulating intent and triggers a guidance window, prompting the user to describe their goal in broader terms or select from suggested query templates.
- **Prolonged inactivity following a system response:** The system records the timestamp of each response and continuously monitors subsequent mouse and keyboard events. If no meaningful interaction is detected within 40 seconds after a response, the system interprets this as a potential uncertainty or difficulty comprehending the output and proactively asks whether further explanation is needed. Meaningful interactions are defined as actions that reflect deliberate progress in query construction, including typing in the input field, clicking on interface elements (e.g., executing SQL, expanding schema nodes, or selecting recommended options), and scrolling through query results. In contrast, actions such

as idle mouse movement, unfocused clicks outside interactive regions, or brief cursor hovering are treated as non-meaningful and do not reset the inactivity timer. This distinction aims to reduce false positives for users who are genuinely reading or reflecting, while still enabling the system to detect cases where users appear hesitant or disengaged.

- **SQL execution resulting in errors or empty result sets:** When the user executes a SQL query and produces an execution error or an empty table, the monitoring module immediately detects this outcome and triggers a diagnostic guidance window.

As illustrated in Figure 1, *QueryGenie* employs a rule-based monitoring module that continuously tracks these signals during user interaction. When a signal is detected, the system proactively intervenes by presenting a guidance window. Figure 4 (B) shows concrete examples of these interventions, in which each signal is associated with a corresponding form of guidance. For example, prolonged inactivity is interpreted as uncertainty or difficulty in understanding the current output; in response, the system prompts the user to indicate whether further explanation is needed. Users may choose to accept or dismiss. If accepted, the system issues a follow-up request on the user’s behalf (e.g., “please explain the SQL step by step”) and generates an appropriate response. This mechanism allows *QueryGenie* to provide timely, context-aware assistance even when users do not explicitly request help, while preserving user agency.

4.2.2 Adaptive Content Representation. Once a user request is determined to be satisfiable, *QueryGenie* immediately performs inference and generates a response. Rather than adopting a fixed presentation format, the system dynamically adapts how response content is structured and displayed to better align with the user cognitive needs (Figure 1 B).

We distinguish three recurring types of user cognitive needs based on observations from our formative study. By analyzing the prompts participants submitted to NL-based querying tools during the walkthroughs, we identified the following categories: (1) **efficiency-oriented**, reflected in direct, action-focused prompts that prioritize obtaining results quickly (e.g., “*show me the recent transactions from last month*”); (2) **explanation-oriented**, reflected in interrogative or clarification-seeking prompts aimed at understanding query logic or diagnosing unexpected results (e.g., “*why does this query return nothing?*”, “*what does this condition mean?*”); and (3) **optimization-oriented**, reflected in corrective or refinement-focused prompts that reference an existing query as a target for improvement (e.g., “*rewrite this query to also filter by region*”, “*can you simplify this?*”). These three need types were consistent across participants and tasks and directly informed the design of *QueryGenie*’s adaptive content representation mechanism.

The classification of each request into one of these three intent types is performed by the decision module illustrated in Figure 1 (B). This module reasons over the current request and conversational context using the following criteria:

- **Efficiency-oriented:** A request is classified as efficiency-oriented when the user’s primary goal is to obtain a query result with minimal overhead. Indicative signals include direct, action-focused phrasing (e.g., “*give me the sales data for*

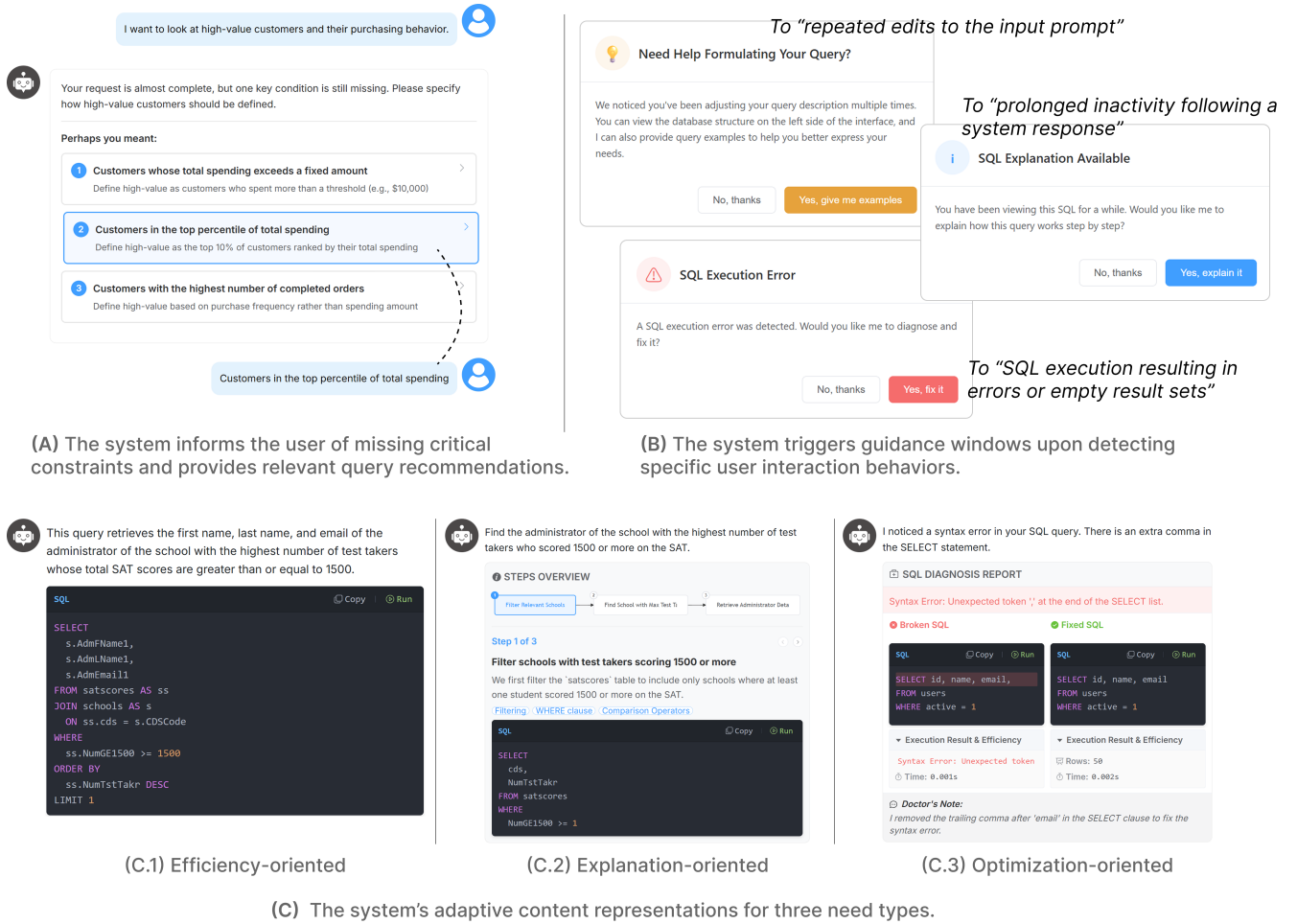


Figure 4: Typical adaptive interaction cases in QueryGenie: (A) The system proactively suggests options to help the user refine the query request; (B) The system detects specific user behaviors and triggers guidance via popup windows; (C) The system dynamically adjusts the representation of content based on the user needs.

last month”) and the absence of interrogative framing (e.g., no “why”, “how”, or “what does this mean”).

- **Explanation-oriented:** A request is classified as explanation-oriented when the user seeks to understand how a query is constructed or why it produces certain results. Indicative signals include interrogative or reflective phrasing (e.g., “why does this return empty results?”, “can you explain this step by step?”), explicit expressions of confusion (e.g., “I don’t understand...”), and cases where the monitoring module has flagged prolonged inactivity.
- **Optimization-oriented:** A request is classified as optimization-oriented when the user aims to improve or refine an existing query. Indicative signals include corrective phrasing (e.g., “rewrite this query to also include...”), explicit references to a prior query as a target for modification rather than as an object of understanding (e.g., “simplify this query”), and requests that introduce new constraints, aggregations, or structural modifications to an existing query.

When multiple signals categories are present, the decision module prioritizes the intent type that is most prominently reflected in the current request, using the broader conversational history as a tiebreaker. Based on the classified intent, the system adaptively switches its content representation, as illustrated in Figure 4 (C).

When the inferred need prioritizes efficiency, the system emphasizes concise outputs by foregrounding executable SQL while minimizing explanatory content (Figure 4 C.1). For explanation-oriented needs, the system augments the generated query with structured, step-by-step explanations that articulate how individual query components are constructed and connected (Figure 4 C.2). Users can navigate across the logical steps of the query through a node-link diagram to inspect the underlying reasoning process. For optimization-oriented needs, the system highlights differences between the original and optimized queries, including both SQL-level changes and differences in execution results (Figure 4 C.3). Through a side-by-side comparison view, users can clearly perceive how specific modifications affect query behavior and outcomes.

4.3 Implementation Details

QueryGenie is implemented as a web-based interactive system. The front-end interface is built using *Vue* and provides a conversational workspace while continuously capturing users' interaction behaviors. The back-end is implemented in *Python* and is responsible for dialogue state management as well as communication with the LLM and the database engine. The front-end and back-end communicate via *Flask*.

As illustrated in Figure 1, the system's adaptive mechanism is implemented through three core modules: two decision modules and one monitoring module. The monitoring module runs continuously throughout user interaction, employing rule-based heuristics to detect specific interaction signals. The two decision modules operate independently, each powered by a separate LLM instance. Rather than functioning as a simple classifier that outputs a category label, each decision module performs a two-step reasoning process within a single LLM call: it first determines the appropriate category (e.g., whether a request is satisfiable or which intent type best characterizes the user's need), and then reasons about how the response content should be structured and generated accordingly. Both decision modules reason by jointly considering the environmental context—including dialogue history, the current query state, and database schema information—alongside the criteria described in subsection 4.2.

5 Evaluation

To evaluate the effectiveness of *QueryGenie*, we conducted a user study with two primary objectives: (1) to compare *QueryGenie* against representative baseline systems in terms of task performance, efficiency, and user experience; and (2) to collect user feedback regarding our adaptive mechanisms. The study is therefore divided into two parts: a system comparison study and an adaptive mechanism evaluation.

5.1 System Comparison Study

We adopted a within-subjects design to compare the performance and user experience across three distinct systems.

5.1.1 Baseline Systems. We compared *QueryGenie* with two representative LLM-based systems:

- *Vanna* [40]: A business solution, which integrates the database environment with a CUI and adopts a purely dialogue-based interaction paradigm, representing the current industrial NLIDB.
- *SQLucid* [37]: An interaction-enhanced interface, representing academic advancements. The system aims to improve query transparency and interaction efficiency by integrating CUI, database visualization, and step-by-step explanation.

To ensure fairness in comparison, all three systems were configured to use the same underlying LLM, *GPT-4o*, thus isolating the impact of the distinct interaction designs from the raw power of the underlying models. The specific interfaces for the baseline systems are provided in Appendix B.

5.1.2 Participants. We recruited 21 participants (**P1–P21**; 6 female, 15 male; aged 19–30) through email and word-of-mouth. We specifically screened for individuals with no formal education or

Table 2: Questionnaire on System Usability (Q1–5) and User Workload (Q6–10), rated on a 7-point Likert scale. The questions was based on the System Usability Scale (SUS) [2] and the NASA Task Load Index (NASA-TLX) [10].

System Usability (1 – strongly disagree, 7 – strongly agree)	
Q1	I think I would like to use this system frequently.
Q2	I found the system unnecessarily complex.
Q3	I thought the system was easy to use.
Q4	I found the various functions in this system were well integrated.
Q5	I felt very confident using the system.
User Workload (1 – very low, 7 – very high)	
Q6	How mentally demanding was the task?
Q7	How hurried or rushed was the pace of the task?
Q8	How successful were you in accomplishing what you were asked to do? (1 – perfect, 7 – failure)
Q9	How much effort did it take to achieve your performance?
Q10	How insecure, discouraged, irritated, and stressed were you?

professional training in SQL to ensure they met the profile of non-expert users. None of the participants had been involved in our preceding formative study.

5.1.3 Tasks. We constructed a small-scale task pool by performing stratified random sampling from the publicly available *BIRD* benchmark [20], focusing on tasks targeting the same database. Specifically, we randomly selected 30 tasks from each of the three difficulty levels defined by the *BIRD* benchmark—simple, moderate, and challenging (detailed definitions are provided in Appendix A)—resulting in a total of 90 unique tasks in the pool.

5.1.4 Procedure. Each participant completed sessions with all three systems to enable within-subject comparisons. For each system, participants began with a brief 3-minute tutorial video introducing core features, followed by a 10-minute free exploration period to familiarize themselves with the interface and interaction modalities, during which support staff were available to answer questions. After familiarization, participants performed 6 tasks of varying complexity (2 simple, 2 moderate, and 2 challenging) using the current system, while task performance and duration were recorded. Immediately after completing the tasks with each system, participants filled out a 7-point Likert-scale questionnaire (Table 2) assessing system usability and cognitive workload. The order of system usage was counterbalanced across participants using a Latin square design to mitigate learning and fatigue effects, ensuring fair and comparable measurements across all systems.

5.1.5 Metrics. With the participants' informed consent, we collected the following key metrics during the experiment:

- **Final Query Accuracy (FQA):** The proportion of tasks correctly completed by each participant, where the final retrieved data must be aligned with the ground truth.
- **Task Completion Time (TCT):** The time (in seconds) taken by each participant to complete each task.
- **Subjective Metrics:** Data collected from the Likert-scale questionnaires filled out by participants after using each system, encompassing metrics related to system usability and user workload.

Table 3: Comparison of final query accuracy (FQA) and average task completion time (TCT) between *QueryGenie* and the baseline systems (*Vanna*, *SQLucid*).

System	FQA	FQA across task difficulties	Avg. TCT (s)	Avg. TCT across task difficulties (s)
<i>QueryGenie</i>	85.7% (108/126)	Simple: 97.6% (41/42) Moderate: 83.3% (35/42) Challenging: 76.2% (32/42)	207 ± 84	Simple: 139 ± 51 Moderate: 187 ± 76 Challenging: 292 ± 112
<i>Vanna</i>	68.3% (86/126)	Simple: 81.0% (34/42) Moderate: 64.3% (27/42) Challenging: 60.0% (25/42)	218 ± 179	Simple: 113 ± 91 Moderate: 188 ± 148 Challenging: 352 ± 183
<i>SQLucid</i>	76.2% (96/126)	Simple: 92.9% (39/42) Moderate: 71.4% (30/42) Challenging: 64.3% (27/42)	232 ± 93	Simple: 162 ± 63 Moderate: 247 ± 88 Challenging: 287 ± 94

- **Interaction Log:** The system automatically recorded the complete interaction history between each participant and the system.

5.2 Comparison Results

We compared the three systems across three key dimensions: task performance, system usability, and user cognitive load. As part of our preliminary analysis, we conducted a Kruskal-Wallis H test [17] on participants' ratings across the systems, followed by a Tukey HSD post-hoc test [38] to examine pairwise differences.

5.2.1 Task Performance. As shown in Table 3, *QueryGenie* exhibited a commanding lead in **FQA**, outperforming both baseline systems across all difficulty levels. This advantage was further amplified by task complexity: when confronted with challenging tasks, *QueryGenie* maintained a high accuracy of 76.2%, while the accuracy of the two baseline systems dropped to around 65%. This strongly validates that *QueryGenie*'s mechanisms for human-AI collaboration are crucial for ensuring the high reliability of queries in complex scenarios.

Regarding the **TCT**, the overall duration across the three systems was comparable, with *QueryGenie* showing a marginal edge in efficiency. Analyzing by difficulty level reveals a more nuanced picture. For simple tasks, *Vanna* demonstrated the fastest completion speed (113 ± 91 s), leveraging the brevity of its purely conversational approach. *QueryGenie*'s **TCT** for simple tasks (139 ± 51 s) was comparatively higher, which can be attributed to the overhead introduced by its adaptive mechanisms: when user intent is already clear and query logic is straightforward, proactive scaffolding behaviors, such as offering step-by-step explanations or suggested query options, may add interaction cost that outweighs their benefit in low-complexity scenarios. Nevertheless, *QueryGenie*'s substantially lower standard deviation indicates more consistent and predictable performance, even on simple tasks.

As task difficulty increased, however, the limitations of purely reactive systems became apparent. *Vanna*'s **TCT** rose sharply for challenging tasks (352 ± 183 s), reflecting the high cost of repeated failed attempts without system guidance. In contrast, *QueryGenie* achieved the lowest **TCT** in challenging tasks (292 ± 112 s), demonstrating that its adaptive framework effectively guides users through debugging and correction in complex scenarios. *SQLucid*

recorded the highest **TCT** for both simple and moderate tasks, suggesting that its fixed workflow introduced unnecessary overhead.

5.2.2 System Usability. As summarized in Figure 5, *QueryGenie* received significantly higher usability ratings than both baseline systems on most system usability measures. In particular, statistically significant differences were observed in willingness to use (**Q1**), perceived ease of use (**Q3**), functional integration (**Q4**), and user confidence (**Q5**) under the Kruskal-Wallis test, followed by pairwise comparisons.

Participants reported a markedly higher willingness to use *QueryGenie* (**Q1**: 6.29 ± 0.70) compared to *Vanna* (4.71 ± 0.45 , $p < .01$) and *SQLucid* (5.29 ± 0.70 , $p < .05$). Similarly, ratings for functional integration (**Q4**) and confidence in using the system (**Q5**) showed strong and consistent advantages for *QueryGenie*, with all pairwise comparisons reaching statistical significance. No significant differences were found among the three systems for perceived complexity (**Q2**), suggesting that the observed usability gains were not driven by reduced feature richness, but rather by how system functionality was organized and presented during interaction. Taken together, these results indicate that participants perceived *QueryGenie* as more coherent, trustworthy, and suitable for repeated use than the baseline systems.

5.2.3 User Cognitive Load. *QueryGenie* also demonstrated significant advantages in reducing user-reported workload, as shown in Figure 5. Participants reported significantly lower mental demand (**Q6**), higher perceived task success (**Q8**), and lower required effort (**Q9**) when using *QueryGenie* compared to both baselines.

Specifically, mental demand ratings for *QueryGenie* (**Q6**: 2.14 ± 0.64) were significantly lower than those for *Vanna* (3.57 ± 0.73 , $p < .01$) and *SQLucid* (3.14 ± 0.64 , $p < .05$). Perceived task success (**Q8**) and effort (**Q9**) followed a similar pattern, with *QueryGenie* consistently outperforming both baseline systems across pairwise comparisons. No statistically significant differences were observed for time pressure (**Q7**) or negative affect (**Q10**), suggesting that the workload benefits of *QueryGenie* were primarily associated with reduced cognitive effort and increased confidence in task outcomes, rather than changes in pacing or emotional stress.

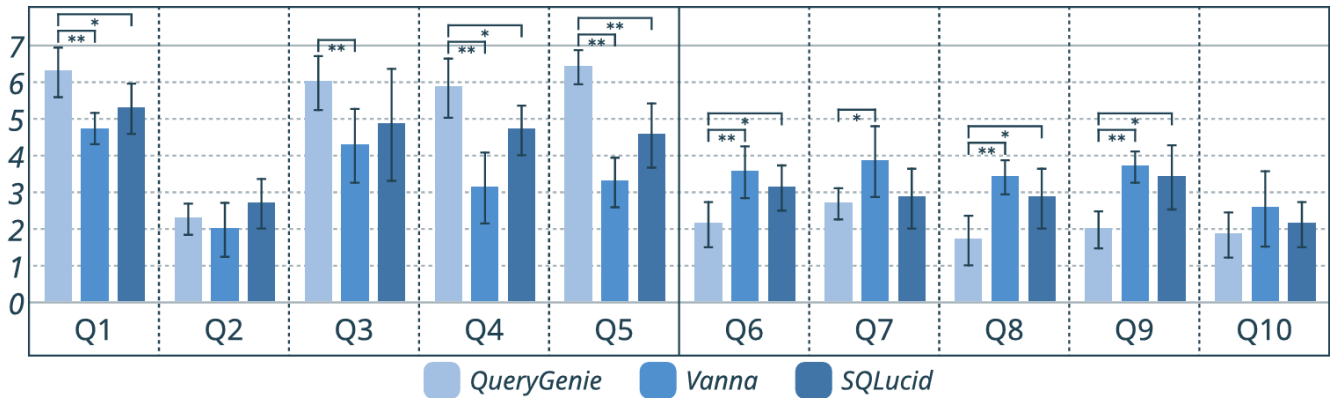


Figure 5: Statistical comparison of *QueryGenie* (Q) against *Vanna* (V) and *SQLucid* (S). Bar heights represent mean performance, with error bars indicating standard deviation (Mean $\pm$ S.D.). Asterisks denote statistical significance: * $p < .050$, ** $p < .010$, *** $p < .001$. Detailed numeric results are reported in Table 4.

5.3 Adaptive Mechanism Evaluation

While the system comparison study demonstrates that *QueryGenie* outperforms baseline systems in task performance, usability, and cognitive workload, these quantitative results do not fully explain how users experienced and benefited from the system’s adaptive behaviors during interaction. To better understand this, we extracted all proactive intervention events from the interaction logs, including instances where the system proactively requested additional context upon detecting unsatisfiable queries, as well as interventions triggered by implicit interaction signals (repeated edits, prolonged inactivity, and SQL execution errors). These logs were used to quantitatively characterize the system’s adaptive behaviors during the study. In addition, after completing all task sessions and questionnaires, we conducted individual semi-structured interviews with each participant, each lasting approximately 15–20 minutes and focused exclusively on their experiences with *QueryGenie*. The interview questions were designed to elicit reflections on how the system adjusted its responses during query formulation, error handling, and refinement, as well as how these adjustments influenced participants’ sense of control, understanding of the query process, and confidence in the final results. All interviews were audio-recorded with participants’ consent, subsequently transcribed, and then analyzed through open coding and iterative thematic analysis to identify recurring patterns in participants’ feedback.

5.3.1 Quantitative Analysis of Proactive Interventions. Figure 6 summarizes the average number of proactive intervention triggers per task across the three difficulty levels. Across all 126 tasks, a total of 164 proactive system interventions were recorded. As expected, all four intervention types exhibited a clear upward trend with increasing task difficulty. Among the three implicit interaction signals, *Prolonged Inactivity* was triggered most frequently, averaging 0.12, 0.67, and 1.02 times per task for simple, moderate, and challenging tasks, respectively. This suggests that as task complexity increased, participants paused more often after receiving system responses—likely reflecting heightened uncertainty about the output or greater difficulty in interpreting it. *Repeated Edits* followed a

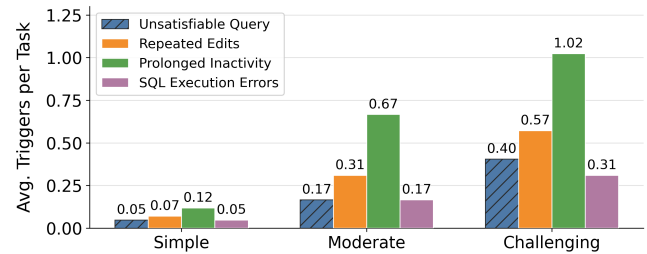


Figure 6: Average number of proactive intervention triggers per task across three difficulty levels.

similar but less pronounced pattern (0.07, 0.31, and 0.57), indicating that participants occasionally struggled to articulate their intent, though less frequently than they exhibited post-response hesitation. *SQL Execution Errors* remained relatively infrequent across all difficulty levels (0.05, 0.17, and 0.31), consistent with the strong SQL generation capability of the underlying *GPT-4o* model. The explicit signal, *Unsatisfiable Query*, was triggered on average 0.05, 0.17, and 0.40 times per task, indicating that participants faced greater difficulty in formulating well-specified queries as task complexity increased, leading the system to proactively solicit clarification.

5.3.2 User Feedback. We identified three prominent themes from the interviews.

Perceived guidance without loss of control. Participants consistently described *QueryGenie*’s adaptive behaviors as providing timely guidance without feeling intrusive or prescriptive. Rather than explicitly instructing users what to do next, *QueryGenie* adjusted its responses based on interaction context, which participants perceived as “nudges” embedded within the ongoing dialogue. P2 noted that “instead of dictating what I must do, it [*QueryGenie*] guides the conversation and lets me know what to try next.” Similarly, P6 remarked that “the system felt like it was following my thinking, not interrupting it. When I was unsure, it gave more hints, and when I was confident, it stayed out of the way.” This adaptive subtlety helped

participants maintain a sense of control over the query construction process while still benefiting from system support.

Growing trust through consistent adaptation. Another salient theme was the development of trust over time as participants observed consistent and predictable system adaptations. Participants noted that as they interacted with *QueryGenie* across multiple tasks, they developed expectations about how the system would respond to incomplete, ambiguous, or incorrect inputs. This predictability made the interaction feel more reliable and reduced hesitation in experimenting with refinements. Several participants contrasted this experience with purely conversational systems, where responses were perceived as less stable and harder to anticipate. In *QueryGenie*, adaptive adjustments were viewed as grounded in the current interaction context rather than arbitrary changes in system behavior. As P15 remarked, “After a few tasks, I knew what the system would help me with, so I felt more confident trying things out.”

Enhanced awareness of query logic and errors. Participants also reported that *QueryGenie*’s adaptive responses improved their awareness of query logic and helped them more easily identify errors during query construction. Instead of merely returning incorrect results or generic failure messages, the system adjusted its response structure and explanatory depth to reflect potential issues in users’ requests. These adaptive changes prompted participants to reconsider their assumptions and better understand why a query did not behave as expected.

In summary, these feedback suggest that *QueryGenie*’s adaptive mechanisms did not merely improve efficiency or accuracy, but also shaped how users reasoned about the query process, managed errors, and built confidence during interaction. These qualitative findings complement the quantitative results by explaining how adaptive interaction design contributes to improved performance and user experience in complex query scenarios.

6 Discussion

6.1 Beyond One-Shot: Querying as Co-evolving Sense-making

Traditional research on NL-to-SQL translation has largely framed it as a one-shot, transactional process [4, 14, 44, 49], assuming users begin with a clear and stable intent. In contrast, our findings from non-expert users reveal database query to be a co-evolving process of sense-making, where initial intent is often vague, unstable, or even absent. Users frequently face a “cold-start” dilemma: uncertain of what to ask and lacking the criteria to evaluate potential queries. Systems that respond only to explicit requests amplify this dilemma, placing the full burden of intent articulation and problem diagnosis on the user. Effective support, therefore, cannot presuppose proactive user guidance; instead, it must actively scaffold the early stages of exploratory intent formation.

QueryGenie addresses this through adaptive initiative. By detecting signs of user uncertainty or early exploration, the system proactively structures the interaction—offering explanations, suggestions, or alternative perspectives. This initiates a co-evolutionary loop: the system’s outputs (queries, results, explanations) actively shape the user’s understanding of the data space and query logic, while the user’s evolving responses and behaviors guide the system’s subsequent focus and strategy. Thus, the interaction moves

beyond translating a predefined request toward collaboratively constructing its meaning.

We argue that supporting non-expert querying requires a fundamental paradigm shift—from one-shot translation to interactive sense-making. This shift yields key implications for HCI design: 1) *Design for Uncertainty*: Treat user ambiguity not as an error state but as a central, expected condition of interaction. 2) *Embed Adaptive Scaffolding*: Design systems that can modulate their level of initiative and guidance based on real-time assessment of user understanding and progress. 3) *Frame Outputs as Contributions to Shared Understanding*: Present queries, results, and explanations not as final artifacts, but as propositions that advance a shared, iterative inquiry. By centering the co-evolution of user intent and system support, this work advocates for interaction paradigms that are more responsive to the cognitive and exploratory realities of data inquiry.

6.2 Designing Initiative: Negotiating Leadership in Collaborative Querying

A core design question emerging from our study is how and when an AI system should take the lead in a collaborative query-building session. Our findings reveal that users’ need for system initiative is dynamic and context-dependent. During phases of uncertainty or struggle such as formulating an initial goal or debugging a complex query, users welcome and expect proactive guidance. Conversely, when they have established a clear mental model and direction, excessive system intervention can feel intrusive and can undermine their sense of control and competence. Thus, effective design is not about choosing a fixed locus of control, but about enabling fluid transitions along a spectrum from user-led to system-guided interaction.

Our work demonstrates that this fluidity must be driven by a continuous, multi-modal assessment of user state. *QueryGenie* integrates explicit conversational instructions with implicit behavioral signals (e.g., hesitation, repetition, inactivity) to infer the user’s current cognitive and task context. Initiative, manifested in the form of suggestions, explanations, or structural prompts, is then modulated in real-time. This approach reframes initiative from a static configuration to a context-sensitive, ongoing negotiation between the user and the system.

Crucially, proactive behavior risks being perceived as overbearing. To preserve user agency and trust, every proactive act in *QueryGenie* is coupled with a transparent rationale (e.g., “I’m suggesting this because you’ve revised this clause twice”) and is presented as a non-binding choice. The user retains the ultimate authority to accept, modify, or ignore the input. This aligns with a fundamental design principle: the goal of system initiative is not to replace user decision-making, but to scaffold it—providing timely support while keeping the user firmly in the loop.

Looking forward, we identify key open challenges for the design of adaptive initiative: developing more nuanced models of user mental state and task progression, and designing proactive strategies that are simultaneously adaptive, transparent, and respectful of user autonomy. Achieving this balance is essential for building collaborative systems that are both powerfully supportive and genuinely empowering.

6.3 Coordinating Conversational and Graphical Interaction for Cognitive Support

A central design challenge in LLM-based query systems is how to effectively orchestrate multiple interaction modalities to support the user's cognitive workflow. While prior systems often privilege one modality, e.g., conversational interfaces for flexible expression or graphical interfaces for structured overview, our findings suggest that for non-experts, a tightly coordinated integration of both is critical. This coordination directly addresses the evolving demands of query construction, which requires both exploratory articulation and structured sensemaking.

In *QueryGenie*, natural language conversation provides a low-friction channel for intent exploration, allowing users to articulate, refine, and reflect on goals dynamically. However, relying solely on conversation places a high burden on the user's working memory to track query structure, system interpretations, and intermediate results across turns. To offload this cognitive overhead, the system complements conversation with persistent graphical representations. These visuals externalize and stabilize the LLM's outputs, such as the evolving query structure, result previews, or explanation trails, transforming ephemeral dialogue into inspectable, manipulable artifacts.

Crucially, the two modalities are designed not as parallel channels but as interdependent components of a unified interaction loop. Conversation drives the process forward, while graphics ground and crystallize the system's understanding. Visual elements are embedded within and updated by the conversational flow, allowing users to fluidly shift between expressing intent in natural language and examining its structured counterpart. This creates a hybrid experience where linguistic exploration and visual grounding reinforce each other.

Our work underscores the broader design potential of modality coordination in LLM-driven interfaces. For open-ended, cognitive-intensive tasks like query formulation, conversation enables exploratory intent formation, while graphics support structural understanding and state persistence. This paradigm of continuous coupling between expression and externalization offers a promising template for designing systems that support evolving cognition in complex, iterative tasks—where the user's understanding develops through interaction, not in isolation.

6.4 Limitations and Future Work

This work has several limitations that point to concrete opportunities for future HCI research. First, our findings are based on a formative study ($N = 8$) and a controlled experiment ($N = 21$). While the results are positive and consistent, studies with larger, more diverse participant pools are needed to strengthen the generalizability of our conclusions regarding performance, cognitive load, and user experience. Second, our evaluation captures users' behavior and experience within a short-term, task-oriented setting. We have not yet observed how users' strategies, mental models, and expectations of system initiative evolve over extended, repeated use. Longitudinal studies are essential to understand whether and how such adaptive systems support sustained learning, skill development, and changing patterns of reliance over time. Third, the adaptive mechanisms in *QueryGenie* rely on heuristic rules derived

from observable interaction cues. While effective in our study, these heuristics are an approximation of user cognitive state. Future work should investigate more sophisticated, potentially learning-based user modeling techniques that could enable finer-grained, personalized adaptation. A key open challenge is to enhance this adaptivity while preserving system transparency and user understandability. Finally, although grounded in database querying, the core principles of this work—adaptive mixed-initiative interaction and tightly coupled conversational-graphical scaffolding—address fundamental HCI challenges in supporting open-ended, cognitive-intensive tasks. Future research should explore transferring and tailoring this framework to other domains where users navigate uncertainty, evolving goals, and complex information spaces, such as exploratory data analysis, learning platforms, or clinical decision support systems.

7 Conclusion

This paper introduced *QueryGenie*, an adaptive interactive system for database querying. *QueryGenie* advances human-AI collaboration in query formulation by integrating adaptive mechanisms that dynamically modulate the system's level of initiative and the presentation of information in response to user instructions and implicit interaction behaviors. This approach is designed to reduce cognitive load and flexibly support users' evolving information needs throughout an interactive query construction session. A controlled user study ($N = 21$) demonstrates that, compared to representative baseline systems, *QueryGenie* significantly improves task performance and efficiency, reduces measurable cognitive load, and is perceived as providing a more intelligent and supportive interaction experience. These findings highlight the value of adaptive interaction for complex information tasks and contribute a design framework for systems that collaboratively guide and refine user intent. Future work will explore long-term adoption and adaptation across diverse user expertise levels and database complexities.

Acknowledgments

We gratefully acknowledge the anonymous reviewers for their insightful feedback. This research was supported by the National Natural Science Foundation of China (No. 62372298), the Shanghai Engineering Research Center of Intelligent Vision and Imaging, the Shanghai Frontiers Science Center of Human-centered Artificial Intelligence (ShangHAI), and the MoE Key Laboratory of Intelligent Perception and Human-Machine Collaboration (KLIP-HuMaCo).

References

- [1] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N Bennett, Kori Inkpen, et al. 2019. Guidelines for human-AI interaction. In *Proceedings of the 2019 chi conference on human factors in computing systems*. 1–13.
- [2] John Brooke et al. 1996. SUS-A quick and dirty usability scale. *Usability evaluation in industry* 189, 194 (1996), 4–7.
- [3] Tommaso Calò, Andrea Sillano, and Luigi De Russis. 2025. MorphGUI: Real-time GUIs customization with large language models. *International Journal of Human-Computer Studies* (2025), 103695.
- [4] DongHyun Choi, Myeong Cheol Shin, EungGyun Kim, and Dong Ryeol Shin. 2021. Ryansql: Recursively applying sketch-based slot fillings for complex text-to-sql in cross-domain databases. *Computational Linguistics* 47, 2 (2021), 309–332.
- [5] DataLine.app. 2024. <https://dataline.app/>.
- [6] Gerhard Fischer. 2001. User modeling in human-computer interaction. *User modeling and user-adapted interaction* 11, 1 (2001), 65–86.
- [7] Vasco Furtado, Jorge Araujo, Elizabeth S Furtado, Rafael Bomfim, Vitória Silva, Nickolas Mororó, Lucas Vasconcelos, Cecília Carvalho, Marcos Severo, Juliana

- Torres, et al. 2025. Assessing the User Experience of an LLM-Based Conversational Assistant in Diabetes Mellitus Care. *Journal of Healthcare Informatics Research* (2025), 1–38.
- [8] Tong Gao, Mira Dontcheva, Eytan Adar, Zhicheng Liu, and Karrie G Karahalios. 2015. Datatone: Managing ambiguity in natural language interfaces for data visualization. In *Proceedings of the 28th annual acm symposium on user interface software & technology*. 489–500.
- [9] Izzeddin Gür, Semih Yavuz, Yu Su, and Xifeng Yan. 2018. Dialsq: Dialogue based structured query generation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1339–1349.
- [10] Sandra G Hart. 1986. NASA task load index (TLX). (1986).
- [11] Gary G Hendrix, Earl D Sacerdoti, Daniel Sagalowicz, and Jonathan Slocum. 1978. Developing a natural language interface to complex data. *ACM Transactions on Database Systems (TODS)* 3, 2 (1978), 105–147.
- [12] Zijin Hong, Zheng Yuan, Qinggang Zhang, Hao Chen, Junnan Dong, Feiran Huang, and Xiao Huang. 2024. Next-Generation Database Interfaces: A Survey of LLM-based Text-to-SQL. *arXiv preprint arXiv:2406.08426* (2024).
- [13] Eric J Horvitz, John S Breese, David Heckerman, David Hovel, and Koos Rommelse. 2013. The Lumiere project: Bayesian user modeling for inferring the goals and needs of software users. *arXiv preprint arXiv:1301.7385* (2013).
- [14] Binyuan Hui, Xiang Shi, Ruiying Geng, Binhua Li, Yongbin Li, Jian Sun, and Xiaodan Zhu. 2021. Improving text-to-sql with schema dependency learning. *arXiv preprint arXiv:2103.04399* (2021).
- [15] Anthony Jameson. 2007. Adaptive interfaces and agents. In *The human-computer interaction handbook*. CRC press, 459–484.
- [16] Daeun Jeong, Sungbok Shin, and Jongwook Jeong. 2025. Conversation Progress Guide: UI system for enhancing self-efficacy in conversational AI. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–11.
- [17] William H Kruskal and W Allen Wallis. 1952. Use of ranks in one-criterion variance analysis. *Journal of the American statistical Association* 47, 260 (1952), 583–621.
- [18] Pat Langley. 1999. User modeling in adaptive interface. In *UM99 User Modeling: Proceedings of the Seventh International Conference*. Springer, 357–370.
- [19] Fei Li and Hosagrahar V Jagadish. 2014. NaLIR: an interactive natural language interface for querying relational databases. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*. 709–712.
- [20] Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2024. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems* 36 (2024).
- [21] Katie Li, Ashutosh Tiwari, Jeffrey Alcock, and Pablo Bermell-Garcia. 2016. Categorization of visualisation methods to support the design of Human-Computer Interaction Systems. *Applied ergonomics* 55 (2016), 85–107.
- [22] Yuntao Li, Bei Chen, Qian Liu, Yan Gao, Jian-Guang Lou, Yan Zhang, and Dongmei Zhang. 2020. "What Do You Mean by That?" A Parser-Independent Interactive Approach for Enhancing Text-to-SQL. *arXiv preprint arXiv:2011.04151* (2020).
- [23] Brian Y Lim, Anind K Dey, and Daniel Avrahami. 2009. Why and why not explanations improve the intelligibility of context-aware intelligent systems. In *Proceedings of the SIGCHI conference on human factors in computing systems*. 2119–2128.
- [24] Xuanhui Liu, Tim Rietz, and Alexander Maedche. 2025. Conversational versus graphical user interfaces: the influence of rational decision style when individuals perform decision-making tasks repeatedly. *Universal Access in the Information Society* 24, 2 (2025), 1157–1172.
- [25] Jiaju Ma, Lei Shi, Kenneth Aleksander Robertsen, and Peggy Chi. 2025. AmbigChat: Interactive Hierarchical Clarification for Ambiguous Open-Domain Question Answering. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*. 1–18.
- [26] Amama Mahmood, Junxiang Wang, Bingsheng Yao, Dakuo Wang, and Chien-Ming Huang. 2025. User interaction patterns and breakdowns in conversing with llm-powered voice assistants. *International Journal of Human-Computer Studies* 195 (2025), 103406.
- [27] Neville Moray, Toshiyuki Inagaki, and Makoto Itoh. 2000. Adaptive automation, trust, and self-confidence in fault management of time-critical tasks. *Journal of experimental psychology: Applied* 6, 1 (2000), 44.
- [28] Arpit Narechania, Adam Fournay, Bongshin Lee, and Gonzalo Ramos. 2021. DIY: Assessing the correctness of natural language to sql systems. In *Proceedings of the 26th International Conference on Intelligent User Interfaces*. 597–607.
- [29] Vidhya Navalpakkam and Elizabeth Churchill. 2012. Mouse tracking: measuring and predicting users' experience of web-based content. In *Proceedings of the SIGCHI conference on human factors in computing systems*. 2963–2972.
- [30] Minju Park, Sojung Kim, Seunghyun Lee, Soonwoo Kwon, and Kyuseok Kim. 2024. Empowering personalized learning through a conversation-based tutoring system with student modeling. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. 1–10.
- [31] S Peng. 2024. The influence of Graphical User Interfaces on human-computer interaction and the impact of organizing software on decision-making process. In *Proceedings of the 4th International Conference on Signal Processing and Machine Learning, Chicago, USA*, Vol. 15. 213–221.
- [32] Kun Qian, Ryan Shea, Yu Li, Luke Kutszik Fryer, and Zhou Yu. 2023. User adaptive language learning chatbots with a curriculum. In *International Conference on Artificial Intelligence in Education*. Springer, 308–313.
- [33] Mohammad Safaei and Ehsan Ghafourian. 2022. Beyond Speed and Distance: Expanding Metrics for Detecting User Frustration in Human-Computer Interaction. *International Journal of Advanced Human Computer Interaction* 1, 1 (2022), 1–16.
- [34] Mark W Scerbo. 2018. Theoretical perspectives on adaptive automation. In *Automation and human performance*. CRC Press, 37–63.
- [35] Moritz Spiller, Ying-Hsang Liu, Md Zakir Hossain, Tom Gedeon, Julia Geissler, and Andreas Nürnberger. 2021. Predicting visual search task success from eye gaze data as a basis for user-adaptive information visualization systems. *ACM Transactions on Interactive Intelligent Systems (TiIS)* 11, 2 (2021), 1–25.
- [36] Ben Steichen, Cristina Conati, and Giuseppe Carenini. 2014. Inferring visualization task properties, user performance, and user cognitive abilities from eye gaze data. *ACM Transactions on Interactive Intelligent Systems (TiIS)* 4, 2 (2014), 1–29.
- [37] Yuan Tian, Jonathan K Kummerfeld, Toby Jia-Jun Li, and Tianyi Zhang. 2024. SQLucid: Grounding Natural Language Database Queries with Interactive Explanations. (2024).
- [38] John W Tukey. 1949. Comparing individual means in the analysis of variance. *Biometrics* (1949), 99–114.
- [39] Barbara Tversky. 2018. Multiple models. in the mind and in the world. *Historical Social Research/Historische Sozialforschung. Supplement* 31 (2018), 59–65.
- [40] Vanna.AI. 2024. <https://vanna.ai/>.
- [41] Nicolas Wagner, Matthias Kraus, Wolfgang Minker, David Griol, and Zoraida Callejas. 2025. A survey on multi-user conversational interfaces. *Applied Sciences* 15, 13 (2025), 7267.
- [42] William A Woods. 1973. Progress in natural language understanding: an application to lunar geology. In *Proceedings of the June 4-8, 1973, national computer conference and exposition*. 441–450.
- [43] Mulong Xie, Zhenchang Xing, Sidong Feng, Xiwei Xu, Liming Zhu, and Chunyang Chen. 2022. Psychologically-inspired, unsupervised inference of perceptual groups of GUI widgets from GUI images. In *Proceedings of the 30th ACM joint European software engineering conference and symposium on the foundations of software engineering*. 332–343.
- [44] Xiaojun Xu, Chang Liu, and Dawn Song. 2017. Ssqlnet: Generating structured queries from natural language without reinforcement learning. *arXiv preprint arXiv:1711.04436* (2017).
- [45] Vaishnavi Prakash Yache, Laura Moradbakhti, Irene Neuner, and Tanja Veselinovic. 2025. Predicting affective engagement and mental strain from prosodic speech features. *Frontiers in Psychiatry* 16 (2025), 1656292.
- [46] Ziyu Yao, Yu Su, Huan Sun, and Wen-tau Yih. 2019. Model-based interactive semantic parsing: A unified framework and a text-to-SQL case study. *arXiv preprint arXiv:1910.05389* (2019).
- [47] JD Zamfirescu-Pereira, Richmond Y Wong, Bjoern Hartmann, and Qian Yang. 2023. Why Johnny can't prompt: how non-AI experts try (and fail) to design LLM prompts. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–21.
- [48] John M Zelle and Raymond J Mooney. 1996. Learning to parse database queries using inductive logic programming. In *Proceedings of the national conference on artificial intelligence*. 1050–1055.
- [49] Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2sql: Generating structured queries from natural language using reinforcement learning. *arXiv preprint arXiv:1709.00103* (2017).

A Task Difficulty Level

In the user study, all tasks provided to participants were sourced from the *BIRD* benchmark. In this benchmark, the difficulty level of each task is rated by annotators following detailed crowdsourcing rules. The specific rating dimensions are as follows:

- **Question Understanding:** On a discrete scale from 1 to 3, annotators assess the ambiguity and difficulty of comprehending the question's intent, with 1 being straightforward, 2 being clear but requiring more thought, and 3 being extremely ambiguous.
- **Knowledge Reasoning:** On a discrete scale from 1 to 3, annotators rate the amount of external knowledge required to map the question to SQL, with 1 indicating no knowledge is required, 2 requiring evidence of external knowledge for

generating SQLs that is easy to understand, and 3 requiring extensive knowledge and much more thoughts.

- **Data Complexity:** Annotators rate the complexity of schema relations and data size that need analyzing on a discrete scale of 1-3, with 1 being a simple schema and data, 2 being complex schema and values understandable through database description files, and 3 being highly complex and difficult to comprehend values and schema even with description files.

- **SQL Complexity:** Annotators rate the syntactic complexity of the target SQL query on a discrete scale of 1-3, with 1 being a simple SQL without many keywords, 2 being more complicated than 1, and 3 being a highly complex SQL with many functions.

Each dimension is considered equally important for annotations. Tasks are then ranked based on these scores and categorized into simple, moderate, and challenging difficulty levels at proportions of 30%, 60%, and 10%, respectively.

B Baseline Systems

Vanna.AI

Training Data

+ New question

Please list the lowest three eligible free rates for students aged 5-17 in continuation schools. (Eligible free rates for students aged 5-17 = 'Free Meal Count (Ages 5-17)' / 'Enrollment (Ages 5-17)'. Continuation schools belongs to frpm.'Educational Option Type)

Please list the lowest three eligible free rates for students aged 5-17 in continuation schools. (Eligible free rates for students aged 5-17 = 'Free Meal Count (Ages 5-17)' / 'Enrollment (Ages 5-17)'. Continuation schools belongs to frpm.'Educational Option Type)

Please list the phone numbers of the direct charter-funded schools that are opened after 2000/1/1.

Please list the lowest three eligible free rates for students aged 5-17 in continuation schools.

You List the names of schools with more than 30 difference in enrollments between K-12 and ages 5-17? Please also give the full street adress of the schools.

SELECT `School Name`, `MailStreet`, `MailCity`, `MailState`, `MailZip` FROM frpm AS T1 INNER JOIN schools AS T2 ON T1.CDSCode = T2.CDSCode WHERE ABS('Enrollment (K-12)' - 'Enrollment (Ages 5-17)') > 30

SCHOOL NAME	MAILSTREET	MAILCITY	MAILSTATE
Alameda County Community	313 West Winton Avenue	Hayward	CA
California School for the Deaf-Fremont	39350 Gallaudet Drive	Fremont	CA
Alameda High	2201 Encinal Avenue	Alameda	CA
Encinal High	210 Central Avenue	Alameda	CA
Island High (Continuation)	1900 Third Street	Alameda	CA
Albany High	603 Key Route Boulevard	Albany	CA
Ocean View Elementary	1000 Jackson Street	Albany	CA
Berkeley High	1980 Allston Way	Berkeley	CA
Castro Valley High	PO Box 2146	Castro Valley	CA
Independent Elementary	PO Box 2146	Castro Valley	CA

CSV Download

Figure 7: The interface of the baseline system, *Vanna*.

SQLucid

Database

Database: aircraft Table: airport

%_change_2007	aircraft_movements	airport	airport_name	domestic_passenge...	freight_metric_tonnes	international_pass...	total_passengers	tr
1.5%	476093	1	London Heathrow	5562516	1397034	61344638	67054745	1:
2.9%	263653	2	London Gatwick	3730963	107702	30431051	34205887	4:
6.0%	160282	3	London Stansted	2343420	197738	19996047	22960304	11:
4.0%	204810	4	Manchester	2943719	141781	18119230	21219195	11:
2.6%	117809	5	London Luton	1320678	40518	8853224	10180734	6:

1-5 of 10

Please input your question.

How many aircrafts are there?

Sure! Please check and modify the explanation below.

Query Explanation

- In table **aircraft**
- Return the number of records

Show SQL

```
SELECT count(*) FROM aircraft
```

Query Result

```
count(*)
```

5

1-1 of 1

GENERATE

PREVIOUS EDIT

NEXT EDIT

Figure 8: The interface of the baseline system, *SQLucid*. (A) Database panel. (B) Question panel, which supports asking questions in natural language. (C) Query Result panel, which displays the query result and the intermediate results of each step. (D) Query Explanation panel, which displays step-by-step explanation of SQL in natural language.

C Complete Statistical Comparison Results

Table 4: The statistical comparison of *QueryGenie* (*Q*) with *Vanna* (*V*) and *SQLucid* (*S*) in the user study. Performance is reported as Mean±S.D. The *Kruskal-Wallis H test* (*KW Test*) was used to compare all three models, followed by pairwise comparisons (*Q* vs. *V* and *Q* vs. *S*). Sig. indicates statistical significance. *p*-values: * $p < .050$, ** $p < .010$, * $p < .001$.**

	<i>QueryGenie</i>	<i>Vanna</i>	<i>SQLucid</i>	<i>KW Test</i>		<i>Q</i> vs. <i>V</i>		<i>Q</i> vs. <i>S</i>	
	Mean±S.D.	Mean±S.D.	Mean±S.D.	<i>p</i> -Value	Sig.	<i>p</i> -Value	Sig.	<i>p</i> -Value	Sig.
Q1	6.29±0.70	4.71±0.45	5.29±0.70	0.0054	**	0.0038	**	0.042	*
Q2	2.29±0.45	2.00±0.76	2.71±0.70	0.24		0.51		0.275	
Q3	6.00±0.76	4.29±1.03	4.86±1.55	0.025	*	0.0074	**	0.21	
Q4	5.86±0.83	3.14±0.99	4.71±0.70	0.0018	**	0.0034	**	0.037	*
Q5	6.43±0.49	3.29±0.70	4.57±0.90	0.0005	***	0.0017	**	0.0037	**
Q6	2.14±0.64	3.57±0.73	3.14±0.64	0.0094	**	0.0070	**	0.029	*
Q7	2.71±0.45	3.86±0.99	2.86±0.83	0.073		0.037	*	0.88	
Q8	1.71±0.70	3.43±0.49	2.86±0.83	0.0060	**	0.0036	**	0.037	*
Q9	2.00±0.53	3.71±0.45	3.43±0.90	0.0029	**	0.0020	**	0.010	*
Q10	1.86±0.64	2.57±1.05	2.14±0.64	0.29		0.18		0.47	